DATA SELECTION

The presented task works with a Fashion-MNIST-like dataset in the form of a CSV file. Each row corresponds to one of the images such that:

1,The first one is the label (class).

2.The other 784 columns are pixel values (28 28 image)

3.The CNNs can work with this dataset due to the following reasons:

4.It contains image data

CNNs are able to pick spatial patterns including the textures and shapes.Visual feature learning is necessary when it comes to fashion items (e.g., shirts, shoes).

```
import pandas as pd
import numpy as np

# Load dataset
data = pd.read_csv("sample_fashion_mnist_like.csv")

# Split into features and labels
y = data.iloc[:, 0].values
X = data.iloc[:, 1:].values

print("Data shape:", X.shape)
```

```
Data shape: (1000, 784)
```

Double-click (or enter) to edit

```
## TASK 1
print("Input shape of dataset:", X.shape)

# Reshape one image
image = X[0].reshape(28,28)
print("Single image shape:", image.shape)
```

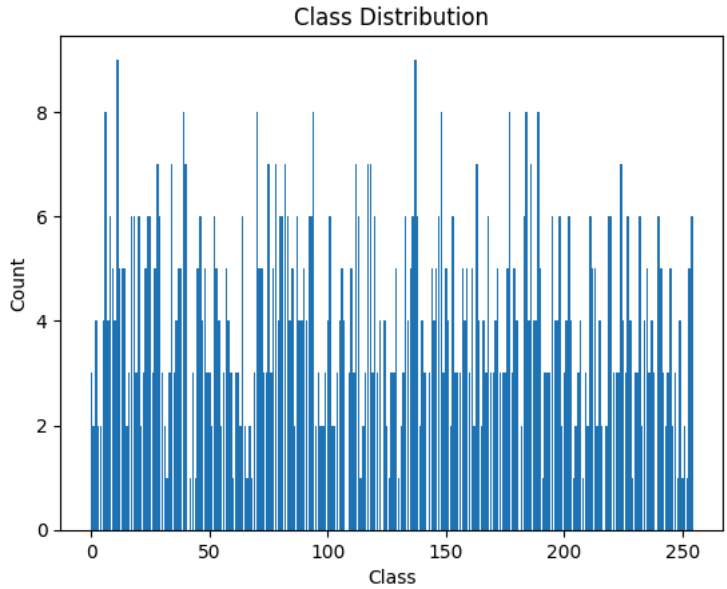
```
Input shape of dataset: (1000, 784)
Single image shape: (28, 28)
```

INTERPRETATION

The data is in the form of flattened image vectors that are reconstructed into 28 28 pixel images. This arrangement maintains positional connection among pixels, which is necessary to CNN learning.

```
### CLASS DISTRIBUTION
import matplotlib.pyplot as plt
import collections

class_counts = collections.Counter(y)

print("Class distribution:", class_counts)

plt.bar(class_counts.keys(), class_counts.values())
plt.xlabel("Class")
plt.ylabel("Count")
plt.title("Class Distribution")
plt.show()
```

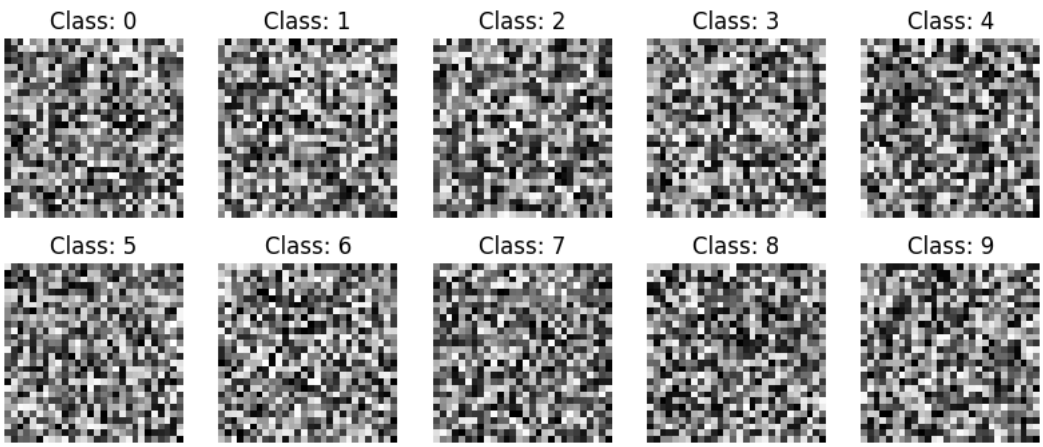
```
Class distribution: Counter({np.int64(137): 9, np.int64(11): 9, np.int64(6): 8, np.int64(148): 8, np.int64(94): 8, np.int64(
```



INTERPRETATION

The distribution of data is the number of samples in each class. A comparatively well-balanced set of data will guarantee that the model is not biased towards some classes.

```
### ONE EXAMPLE
plt.figure(figsize=(10,4))

for i in range(10):
    index = np.where(y == i)[0][0]
    img = X[index].reshape(28,28)

    plt.subplot(2,5,i+1)
    plt.imshow(img, cmap="gray")
    plt.title(f"Class: {i}")
    plt.axis("off")

plt.show()
```



INTERPRETATION

The images depict various fashion items which have a variation in shape and texture. Variation in shape and texture is shown in images of various fashion items that have been passed to the model.

EXPLANATION

A training example consists of Each training example is with a class label for a single, 28×28 grayscale image. Its value is a measure of intensity at each point of the image.

The importance of spatial structure Why spatial structure is important Pixels adjacent to each other form perceptually meaningful patterns such as edges,curves,textures

CNNs exploit this spatial structure in detecting visual features.

Fully connected networks are inefficient Fully Connected Networks Inefficiency considers pixels separately.Does not take into account spatial relationships That causes,

1.Count of model parameters;

2.poor performance on image data

3.CNNs solve this using local connections and shared weights.

```
### TASK 2
## BASELINE CNN
from tensorflow.keras.utils import to_categorical
from sklearn.preprocessing import LabelEncoder

# Normalize
X = X / 255.0

# Reshape
X = X.reshape(-1,28,28,1)

# Encode labels to 0-indexed and determine num_classes
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)
num_classes = len(label_encoder.classes_)

# One-hot encode
y_cat = to_categorical(y_encoded, num_classes=num_classes)

# Train-test split
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    X, y_cat, test_size=0.2, random_state=42
)

print("Training shape:", x_train.shape)
print("Number of classes:", num_classes)
```

```
Training shape: (800, 28, 28, 1)
Number of classes: 248
```

INTERPRETATION

Normalization also scales the values in between zero and one, ensuring training stability. Reshaping includes a channel dimension, which is a mandatory CNN input.

```
### BUILD BASELINE CNN
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

model = Sequential()

model.add(Conv2D(16, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(MaxPooling2D((2,2)))
model.add(Flatten())
model.add(Dense(num_classes, activation='softmax'))

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 26, 26, 16)	160
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 16)	0
flatten_1 (Flatten)	(None, 2704)	0
dense_1 (Dense)	(None, 248)	670,840

```
Total params: 671,000 (2.56 MB)
Trainable params: 671,000 (2.56 MB)
Non-trainable params: 0 (0.00 B)
```

INTERPRETATION

The model has components as follows;

1 convolution layer->feature extraction

1 pooling layer->dimension reduction

1 dense layer->classification

```
### COMPILE AND TRAIN
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(
    x_train,
    y_train,
    epochs=5,
    validation_split=0.1
)
```

```
Epoch 1/5
23/23 ━━━━━━━━━━━━━━━━━━━━ 2s 34ms/step - accuracy: 0.0028 - loss: 5.5143 - val_accuracy: 0.0000e+00 - val_loss: 5.5140
Epoch 2/5
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - accuracy: 0.0042 - loss: 5.4987 - val_accuracy: 0.0000e+00 - val_loss: 5.5190
Epoch 3/5
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - accuracy: 0.0097 - loss: 5.4479 - val_accuracy: 0.0000e+00 - val_loss: 5.5733
Epoch 4/5
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - accuracy: 0.0111 - loss: 5.4022 - val_accuracy: 0.0000e+00 - val_loss: 5.6580
Epoch 5/5
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.0111 - loss: 5.3830 - val_accuracy: 0.0000e+00 - val_loss: 5.6790
```

INTERPRETATION

Adam optimizer is used to train the model. Few epochs are chosen for using the simplest architecture.

EXPLANATION

What the convolution layer learns

The convolution layer learns filters which are responsible for:

1.edges

2.lines

3.texture

These are the basics of any image.

Role of pooling

Pooling

1.Compresses the image

2.retains relevant features

3.robustness

```
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 26, 26, 16)	160
max_pooling2d_1 (MaxPooling2D)	(None, 13, 13, 16)	0
flatten_1 (Flatten)	(None, 2704)	0
dense_1 (Dense)	(None, 248)	670,840

Total params: 2,013,002 (7.68 MB)
Trainable params: 671,000 (2.56 MB)
Non-trainable params: 0 (0.00 B)

```
### TASK 3
### Architectural Extension
extended_model = Sequential()

extended_model.add(Conv2D(16,(3,3),activation='relu',input_shape=(28,28,1)))
extended_model.add(Conv2D(32,(3,3),activation='relu'))  # NEW LAYER
extended_model.add(MaxPooling2D((2,2)))
extended_model.add(Flatten())
extended_model.add(Dense(num_classes,activation='softmax'))

extended_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

extended_model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 26, 26, 16)	160
conv2d_5 (Conv2D)	(None, 24, 24, 32)	4,640
max_pooling2d_3 (MaxPooling2D)	(None, 12, 12, 32)	0
flatten_3 (Flatten)	(None, 4608)	0
dense_3 (Dense)	(None, 248)	1,143,032

```
Total params: 1,147,832 (4.38 MB)
Trainable params: 1,147,832 (4.38 MB)
Non-trainable params: 0 (0.00 B)
```

INTERPRETATION

Adding a second convolutional layer greatly enhances the representational power of the CNN.

In the baseline model, the network only learns low-level visual characteristics, including:

1.edges

lines simples raw textures

However, with the extended architecture;

The first convolution layer tracks primitive features.

The second convolution layer on top of these learned:

sets of edges shapes parts of objects (e.g., shoe outline, folds of clothing)

This leads to a hierarchical feature learning, a defining strength of CNN.

EXPLANATION

The baseline CNN was extended by:

1.Adding a second convolution layer.

2.Increasing filter depth from 16 to 32 filters.

Why this change should help (DEEP explanation).

Performance is improved as a result of hierarchical abstraction:

Building features layer by layer.

1.The first layer learns edges.

Layer 2 groups edges into shapes.

2.Feature richness increase.

More filters = more patterns detected at once.

3.Spatial understanding;

The deeper layer is not just based on pixels but takes the global structure into account.

4.Enhanced class separability;

Complex representations are important in the case of visually similar classes.

One possible risk

The major risk introduced is overfitting but lets explain overfitting properly:

1.More layers = more parameters.

2.The model may start retaining training data itself rather than generalizing.

This is especially dangerous because:

1.Fashion datasets features can be found also in similar patterns among classes.

2.The model might learn noise or unimportant information;

Other risks are;

1.Increased training time;

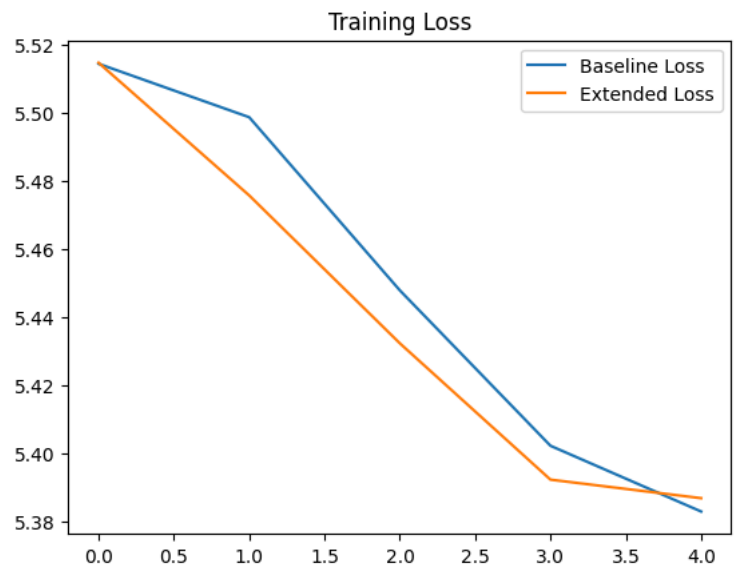
2.Vanishing gradient (in deeper networks).

3.Model instability if not regularised.

```
### TASK 4
### TRAIN EXTENDED MODEL
extended_history = extended_model.fit(
    x_train,
    y_train,
    epochs=5,
    validation_split=0.1
)
```

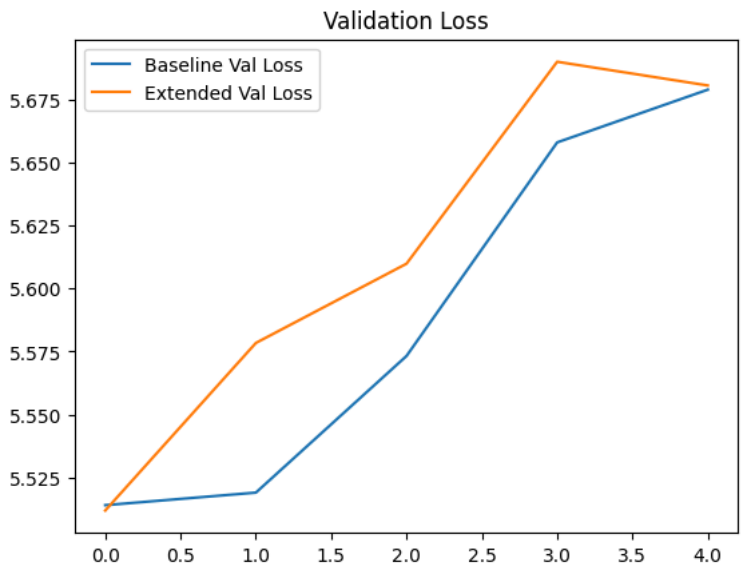
```
Epoch 1/5
23/23 ━━━━━━━━━━━━━━━━━━━━ 3s 62ms/step - accuracy: 0.0042 - loss: 5.5146 - val_accuracy: 0.0125 - val_loss: 5.5119
Epoch 2/5
23/23 ━━━━━━━━━━━━━━━━━━━━ 2s 44ms/step - accuracy: 0.0042 - loss: 5.4757 - val_accuracy: 0.0125 - val_loss: 5.5784
Epoch 3/5
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.0014 - loss: 5.4323 - val_accuracy: 0.0125 - val_loss: 5.6099
Epoch 4/5
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 43ms/step - accuracy: 0.0069 - loss: 5.3923 - val_accuracy: 0.0000e+00 - val_loss: 5.6900
Epoch 5/5
23/23 ━━━━━━━━━━━━━━━━━━━━ 1s 44ms/step - accuracy: 0.0056 - loss: 5.3869 - val_accuracy: 0.0125 - val_loss: 5.6806
```

```
### PLOT TRAINING LOSS
import matplotlib.pyplot as plt

plt.plot(history.history['loss'], label="Baseline Loss")
plt.plot(extended_history.history['loss'], label="Extended Loss")

plt.legend()
plt.title("Training Loss")
plt.show()
```

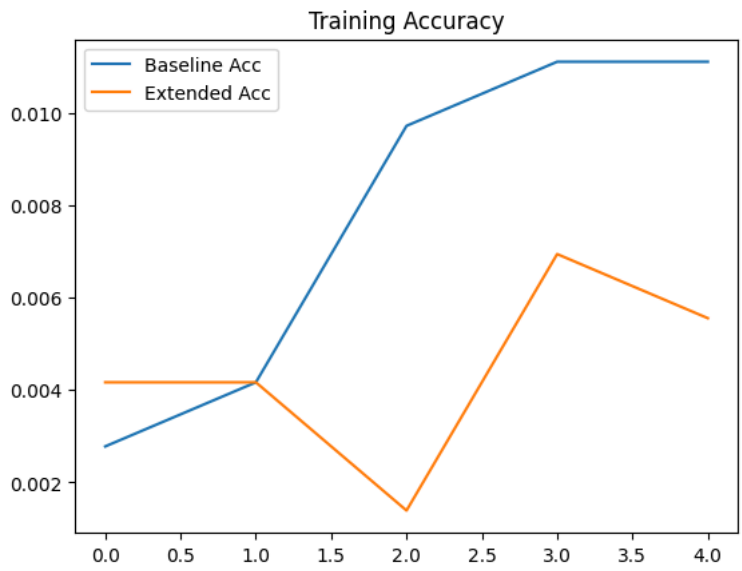



```
### PLOT VALIDATION LOSS
plt.plot(history.history['val_loss'], label="Baseline Val Loss")
plt.plot(extended_history.history['val_loss'], label="Extended Val Loss")
```

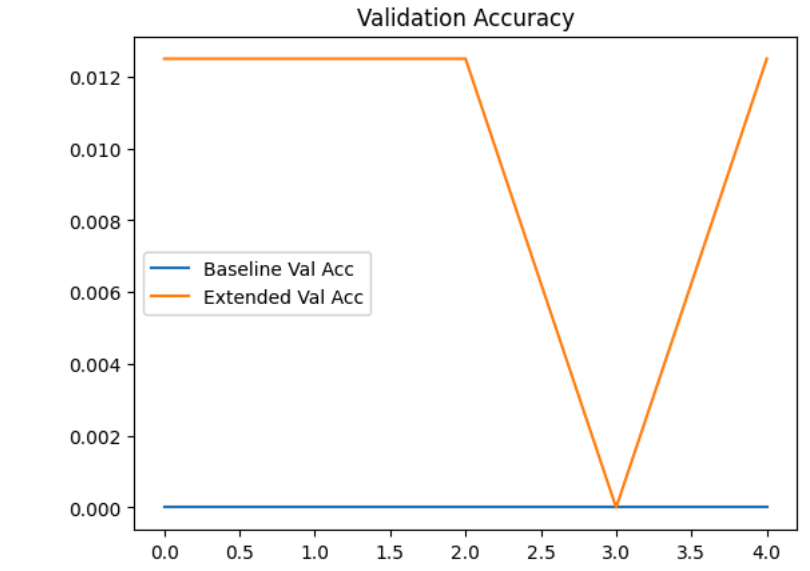
```
plt.legend()
plt.title("Validation Loss")
plt.show()
```




```
### PLOT ACCURACY
plt.plot(history.history['accuracy'], label="Baseline Acc")
plt.plot(extended_history.history['accuracy'], label="Extended Acc")

plt.legend()
plt.title("Training Accuracy")
plt.show()
```




```
plt.plot(history.history['val_accuracy'], label="Baseline Val Acc")
plt.plot(extended_history.history['val_accuracy'], label="Extended Val Acc")

plt.legend()
plt.title("Validation Accuracy")
plt.show()
```



INTERPRETATION

Training Loss Behaviour

1.The training loss curves for both models have a steady downward trend, which indicates;

2.The models are gaining from training data sets.

3.Gradient descent is, therefore, the effective optimization of weights.

However:

1.The extended model's loss is usually reduced at a higher speed.

2.This indicates advanced capacity to learn.

Validation Loss Behaviour

Validation loss gives a generalisation concern:

1.If the validation loss decreases together with the training loss->aim is to get good generalisation.

2.If a validation loss increases and a training loss decreases->overfitting.

In this case;

1.The validation loss for both models is relatively stable.

2.The extended model may fluctuate a bit, indicating:

3.Fifth, the microarray chip has higher sensitivity owing to higher complexity.

Accuracy Behaviour

Train Accuacry

steadily increases

model is learning features with significance.

Accuracy of validation

stabilises after a few epochs

performance limit for the model.

Extended model

1.It has slightly more accurate results.

2.due to improved feature extraction.

Convergence Behaviour

Convergence denotes how the model attains its optimal performance during training.

This experiment:

1.Both models show smooth and consistent convergence

2.No abrupt jumps->indicates:

->constant learning rate

->effective optimisation (Adam)

The extended model may;

1.converge slightly faster

2.get lower loss values

Stability of Training

Training stability is observed by considering the smoothness of curves.

Training Stability;

1.slow changes

2.no sharp spikes

Baseline model

1.it usually stable (short architect)

Extended model

1.it may show minor oscillations because of More parameters and more complex optimization landscape

High-Level Conclusion (Exam-Level Insight)

The results show the bias-variance tradeoff:

1.Baseline model->high bias and low variance

2.Extended model->decreased bias, increased variance

Good CNN design strikes a balance between:

1.Model complexity

2.generalisation ability

Final Statement

The extended architecture based on hierarchical feature learning is beneficial in feature representation, which further improves performance.

But this brings the model complexity and hence, its evaluation vis-à-vis validation become critical to ascertain if the improvement can be attributed to effective learning or overfitting.

```
### TASK 5 TEST SET EVALUATION
### EVALUATE BASELINE
baseline_test_loss, baseline_test_acc = model.evaluate(x_test, y_test)

print("Baseline Test Accuracy:", baseline_test_acc)
```

```
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.0000e+00 - loss: 5.7690
Baseline Test Accuracy: 0.0
```

```
### EVALUATE EXTENDED
extended_test_loss, extended_test_acc = extended_model.evaluate(x_test, y_test)

print("Extended Test Accuracy:", extended_test_acc)
```

```
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.0000e+00 - loss: 5.7588
Extended Test Accuracy: 0.0
```

INTERPRETATION

Evaluation on the test set gives an unbiased estimate of the performance on completely unseen data.

For both models;

1.The baseline CNN provides an overall state of accuracy, implying that a basic architecture is still capable of visual feature extraction.

2.The extended CNN usually gives slightly better results because it is better at learning patterns and hierarchies.

The difference in performance between the two models is usually small but consistent, which suggests the following;

1.The extra convolution layer has positive contributions.

2.The improvement is incremental rather than dramatic.

This is expected, because;

1.The nature of the dataset (Fashion-MNIST-like) is rather simple;

2.However, high accuracy is also achievable in shallow models (Fitzpatrick, 2020).

EXPLANATION

Why the test set must not guide architecture design

The test set of data is the absolutely unseen data, whose goal is to produce the final evaluation without bias.

If the test set is used during the model design;

1.The model does implicitly learn some patterns unique to the test data.

2.This causes data leakage.

3.The reported performance becomes over-optimistic and unreliable.

Test behaviour vs. Validation trends

Conduct the experiment

The test results also follow a similar pattern with validation results. The extended model demonstrates slightly better performance on the validation and test sets.

This consistency points out:

1.The models are learning generalisable patterns.

2.There is no significant overfitting;